Queues and Priority Queues

Chapter 13

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Queue

- Like a line of people
  - First person in line is first person served
  - New elements of queue enter at its back
  - Items leave the queue from its front
- Called FIFO behavior
  - **F**irst **I**n **F**irst **O**ut

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Queue

Abstract Data Type: Queue	
Data	
• A finite number of objects, not necessarily distinct, having the same data type and ordered by when they were added.	
Operations	
Pseudocode	**Description**
`isEmpty()`	Task: Sees whether this queue is empty. Input: None. Output: True if the queue is empty; otherwise false.
`enqueue(newEntry)`	Task: Adds `newEntry` at the back of this queue. Input: `newEntry`. Output: True if the operation is successful; otherwise false.
`dequeue()`	Task: Removes the front of this queue. That is, removes the item that was added earliest. Input: None. Output: True if the operation is successful; otherwise false.
`peekFront()`	Task: Returns the front of this queue. That is, gets the item that was added earliest. The operation does not change the queue. Input: None. Output: The front of the queue.

FIGURE 13-2 Some queue operations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Queue

Operation	Queue after operation
`aQueue` = *an empty queue*	
`aQueue.enqueue(5)`	5
`aQueue.enqueue(2)`	5 2
`aQueue.enqueue(7)`	5 2 7
`aQueue.peekFront()`	5 2 7 (Returns 5)
`aQueue.dequeue()`	2 7
`aQueue.dequeue()`	7

Front

FIGURE 13-2 Some queue operations

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Priority Queue

- Organize data by priorities
  - Example: weekly “to do” list
- Priority value
  - We will say high value ⇨ high priority
- Operations
  - Test for empty
  - Add to queue in sorted position
  - Remove/get entry with highest priority

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Priority Queue

Abstract Data Type: Priority Queue	
Data	
• A finite number of objects, not necessarily distinct, having the same data type and ordered by priority.	
Operations	
Pseudocode	**Description**
`isEmpty()`	Task: Sees whether this priority queue is empty. Input: None. Output: True if the priority queue is empty; otherwise false.
`add(newEntry)`	Task: Adds `newEntry` to this priority queue. Input: `newEntry`. Output: True if the operation is successful; otherwise false.
`remove()`	Task: Removes the entry with the highest priority from this priority queue. Input: None. Output: True if the operation is successful; otherwise false.
`peek()`	Task: Returns the entry in this priority queue with the highest priority. The operation does not change the priority queue. Input: None. Output: The entry with the highest priority.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Priority Queue

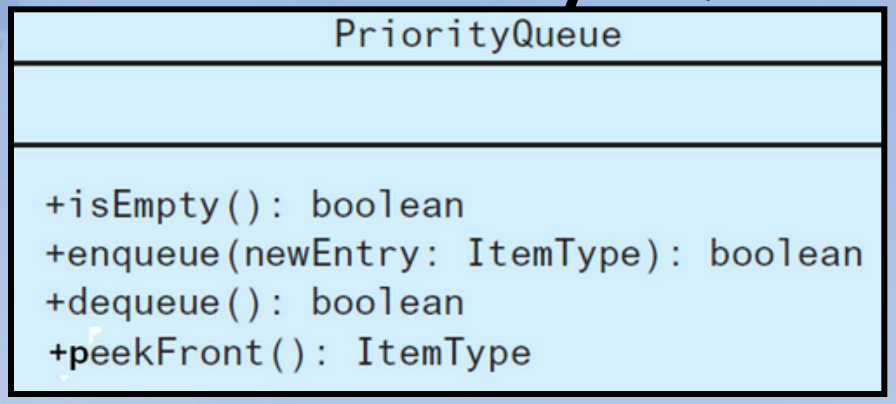
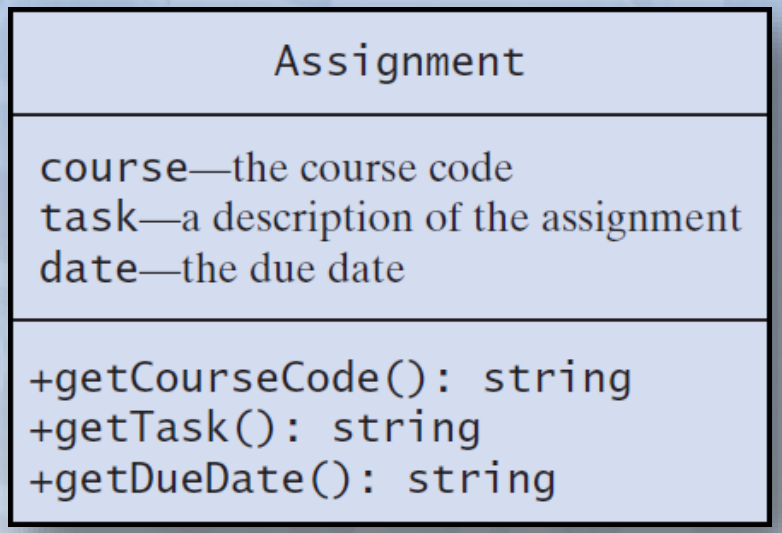


FIGURE 13-4 UML diagram for the class PriorityQueue



Using a priority queue, we can

- organize these assignments in due date order
- task with the earliest due date will have the highest priority
- define a class Assignment of tasks that includes a data field date representing a task's due date

FIGURE 13-5 UML diagram for the class Assignment

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Tracking Your Assignments

```
assignmentLog = a new priority queue using due date as the priority value
project = a new instance of Assignment
essay = a new instance of Assignment
quiz = a new instance of Assignment
errand = a new instance of Assignment
assignmentLog.enqueue(project)
assignmentLog.enqueue(essay)
assignmentLog.enqueue(quiz)
assignmentLog.enqueue(errand)
cout << "I should do the following first: "
cout << assignmentLog.peekFront()
```

Pseudocode to organize assignments, responsibilities

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

- Simulation, a major application area for computers, is models the behavior of both natural and human-made systems
- Mathematicians learned to model events (e.g., the arrival of people) by using techniques from probability theory.
- This statistical information is incorporated into the mathematical model of the system and is used to generate events in a way that reflects the real world.
- The simulation uses these events and is thus called an **event driven simulation**.
- The use of a data file with predetermined event information is common in simulations.
- Problem to solve
  - Approximate average time bank customer must wait for service from a teller
  - Decrease in customer wait time with each new teller added

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

Arrival time	Transaction length
20	6
22	4
23	2
30	3

Sample arrival and transaction times

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

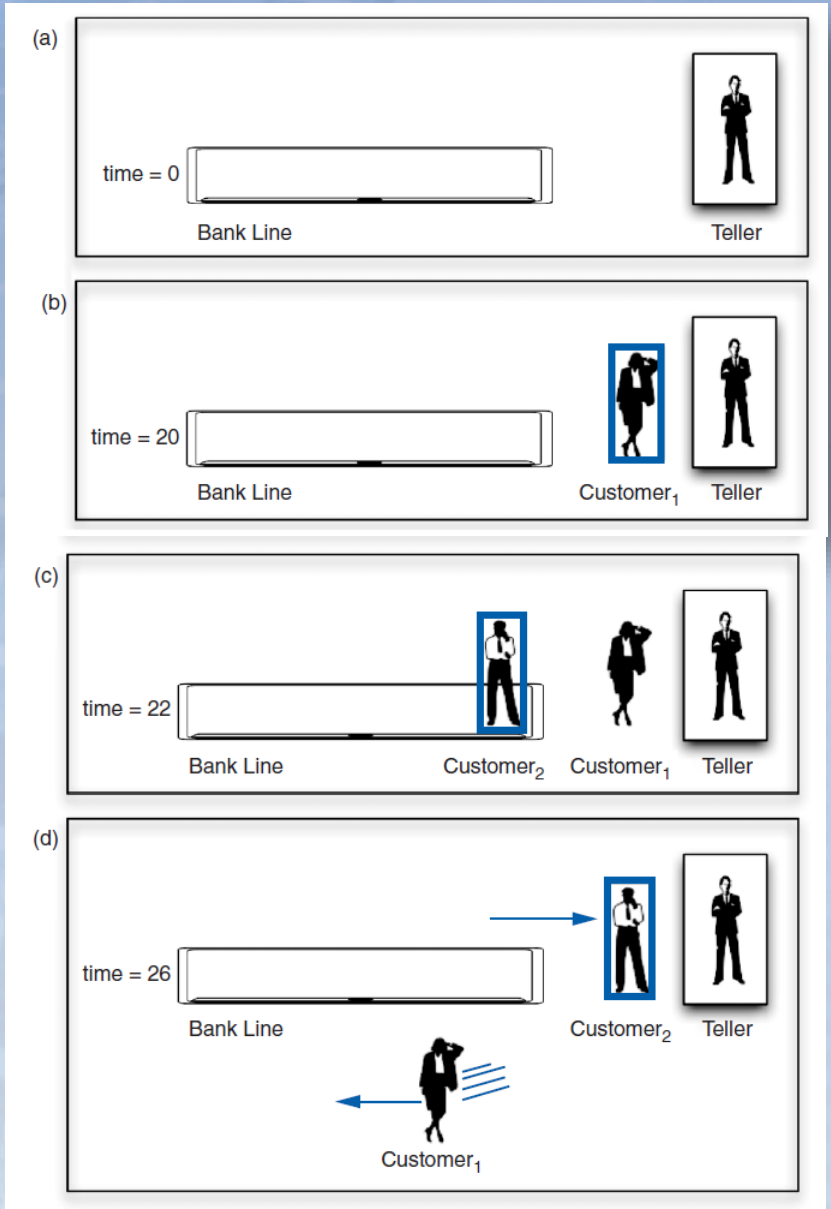


FIGURE 13-6 A bank line at time (a) 0; (b) 20; (c) 22; (d) 26

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

```
Initialize the line to "no customers"
while (events remain to be processed)
{
    currentTime = time of next event
    if (event is an arrival event)
        Process the arrival event
    else
        Process the departure event

    // When an arrival event and a departure event occur at the same time,
    // arbitrarily process the arrival event first
}
```

Pseudocode for an event loop

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

- Time-driven simulation
  - Simulates the ticking of a clock
  - Increment time counter
- Event-driven simulation considers
  - Only the times of certain events,
  - In this case, arrivals and departures
- Event list contains
  - All future arrival and departure events

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

(a) Arrival event

Type	Time	Length
A	20	6

(b) Departure event

Type	Time	Length
D	26	–

FIGURE 13-7 A typical instance of (a) an arrival event; (b) a departure event

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

- Two tasks required to process each event
  - Update the bank line: Add or remove customers.
  - Update the event queue: Add or remove events.
- New customer
  - Always enters bank line
  - Served while at the front of the line

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

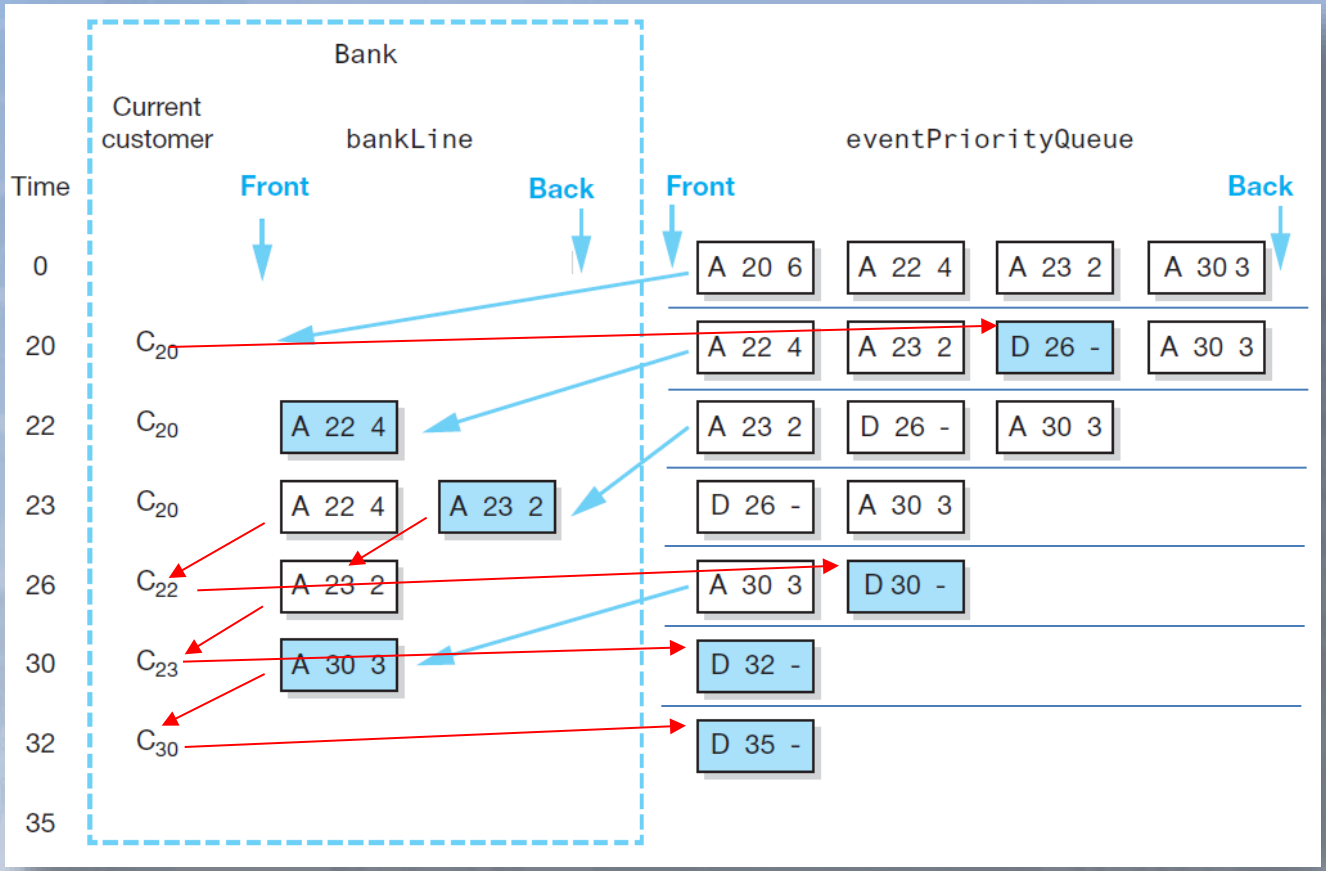


FIGURE 13-8 A trace of the bank simulation algorithm for the data (20, 6), (22, 4), (23, 2), (30, 3)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Position-Oriented and Value-Oriented ADTs

- Position-oriented ATDs
  - Stack, list, queue
- Value-oriented ADTs
  - Sorted list - When a new item is added to a sorted list, the sorted list determines the item's placement according to its value.
    - Unlike a list, a sorted list cannot be told where to place its items.
  - Priority queue - orders its entries according to their priorities

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Position-Oriented and Value-Oriented ADTs

Stack	Queue
isEmpty -	isEmpty
push - insert a new item on top	enqueue - insert a new item at end
pop - removes the most recent item at top of stack	dequeue removes the first item, which is at the front of the queue
peek - retrieves the most recent item, which is at the top of the stack	peekFront - retrieves the first item at the front of the queue

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Position-Oriented and Value-Oriented ADTs

- ADT list operations generalize stack and queue operations
  - getLength
  - insert
  - remove
  - getEntry

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Chapter 13

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Chapter 13

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Queue

```
Queue
+isEmpty(): boolean
+enqueue(newEntry: ItemType): boolean
+dequeue(): boolean
+peekFront(): ItemType
```

FIGURE 13-1 UML diagram for the class Queue

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Queue

```cpp
/** @file QueueInterface.h */
#ifndef QUEUE_INTERFACE_
#define QUEUE_INTERFACE_

template<class ItemType>
class QueueInterface
{
public:
   /** Sees whether this queue is empty.
    @return  True if the queue is empty, or false if not. */
   virtual bool isEmpty() const = 0;

   /** Adds a new entry to the back of this queue.
    @post  If the operation was successful, newEntry is at the
       back of the queue.
    @param newEntry  The object to be added as a new entry.
    @return  True if the addition is successful or false if not. */
    virtual bool enqueue(const ItemType& newEntry) = 0;
```

LISTING 13-1 A C++ interface for queues

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Queue

```
18      virtual bool enqueue(const ItemType& newEntry) = 0;
19
20      /** Removes the front of this queue.
21       @post  If the operation was successful, the front of the queue
22          has been removed.
23       @return  True if the removal is successful or false if not. */
24      virtual bool dequeue() = 0;
25
26      /** Returns the front of this queue.
27       @pre  The queue is not empty.
28       @post  The front of the queue has been returned, and the
29          queue is unchanged.
30       @return  The front of the queue. */
31      virtual ItemType peekFront() const = 0;
32
33      /** Destroys this queue and frees its memory. */
34      virtual ~QueueInterface() { }
35  }; // end QueueInterface
36  #endif
```

LISTING 13-1 A C++ interface for queues

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Applications
Reading a String of Characters

```
// Read a string of characters from a single line of input into a queue
aQueue = a new empty queue
while (not end of line)
{
    Read a new character into ch
    aQueue.enqueue(ch)
}
```

Pseudocode to read a string of characters into a queue.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Applications
Recognizing a Palindrome

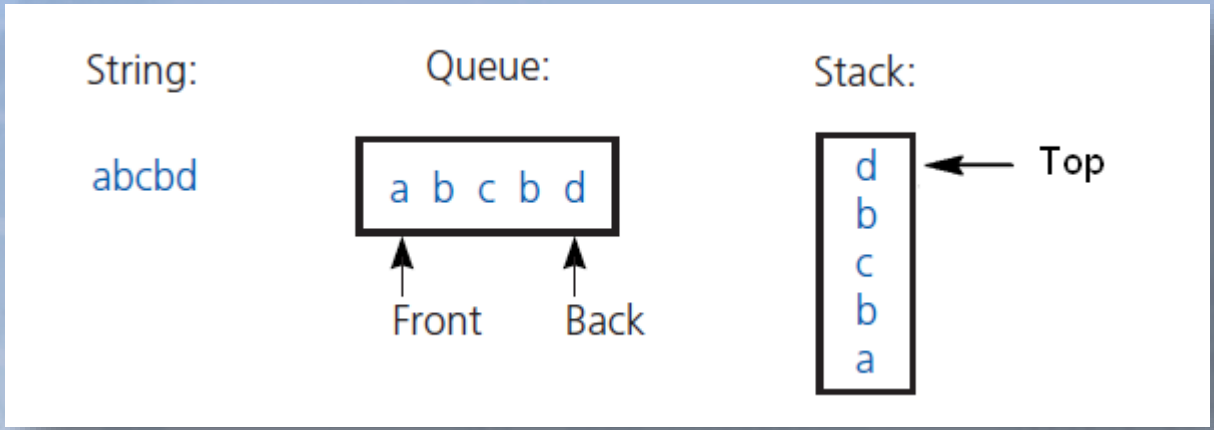


FIGURE 13-3 The results of inserting the characters a, b, c, b, d into both a queue and a stack

- Remove characters from front of queue, top of stack
- Compare each pair removed
- If all pairs match, string is a palindrome

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Tracking Your Assignments

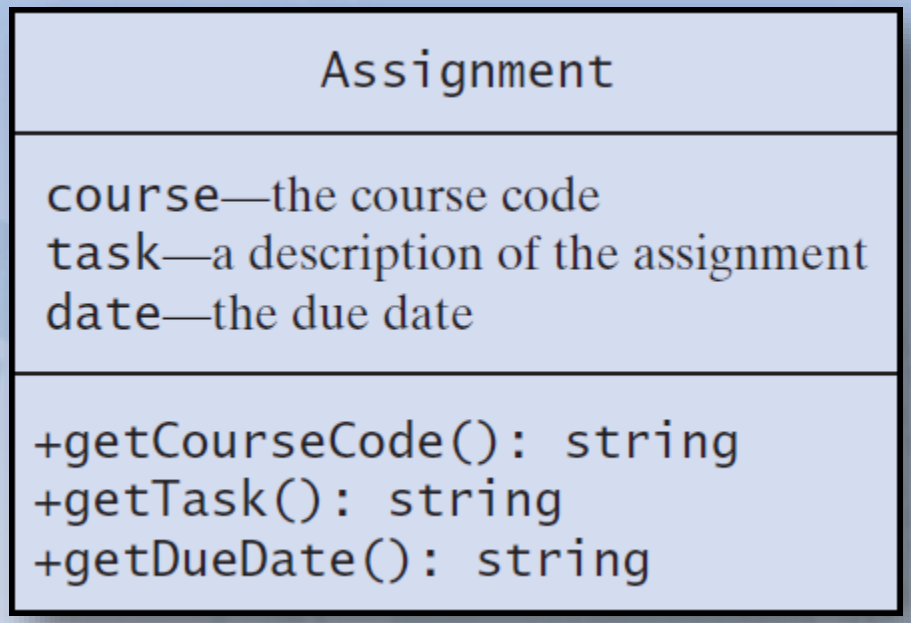


FIGURE 13-5 UML diagram for the class Assignment

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Application: Simulation

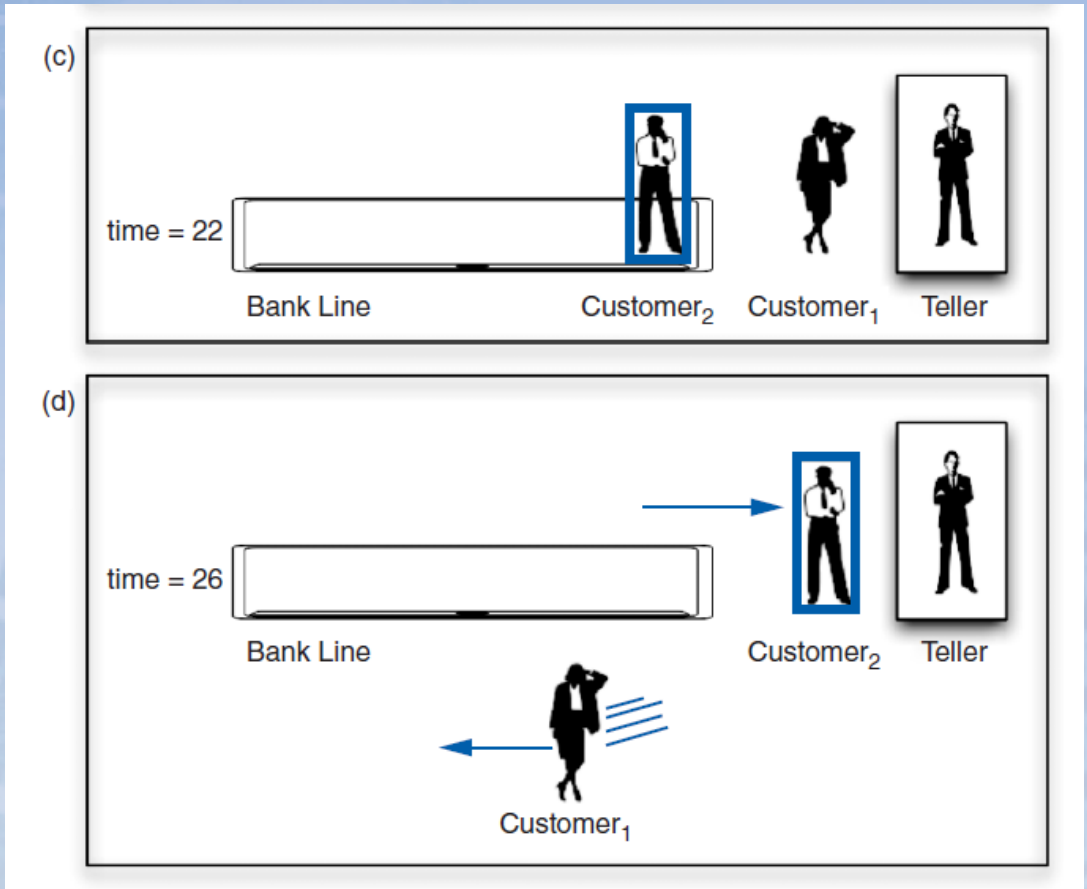


FIGURE 13-6 A bank line at time (a) 0; (b) 20; (c) 22; (d) 26

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Position-Oriented and Value-Oriented ADTs

- Comparison of stack and queue operations
  - isEmpty - see if any items exist in the ADT
  - push and enqueue - insert a new item into one end (the top and back, respectively) of the ADT
  - pop and dequeue - removes the most recent item, which is at the top of the stack, and dequeue removes the first item, which is at the front of the queue
  - peek and peekFront - peek operation retrieves the most recent item, which is at the top of the stack, and peekFront retrieves the first item at the front of the queue

© 2017 Pearson Education, Hoboken, NJ. All rights reserved